



L3 informatique

Course Notes
Software Engineering

Course 7
Component Diagram

Presented by
Mr KHELIFA N.

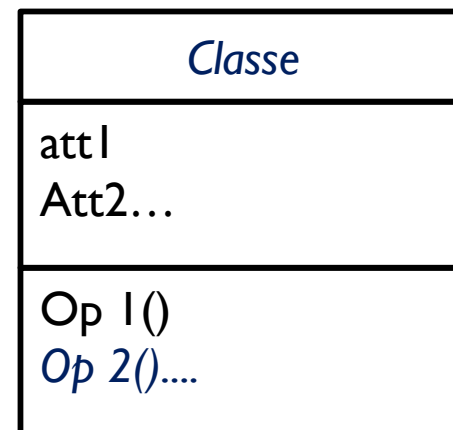
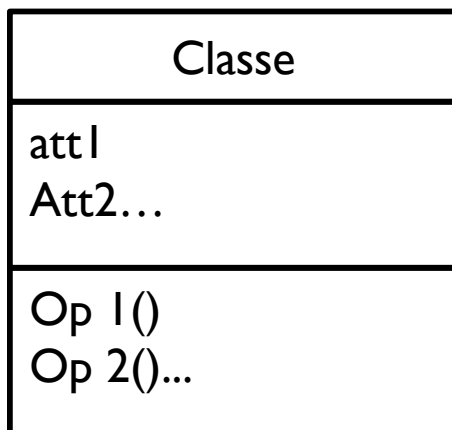
Reminder of Definitions

- **Some Concepts about Classes**

- **Attribute:** A named property of a classifier that describes a range of values that instances of the classifier may assume.
- **Operation:** A service that can be requested from any object of the class.
- **Operation signature:** The signature of an operation consists of its name and all its parameters (including the return parameter, which defines the return type of the operation).
- **Method =** The implementation of an operation.

Abstract Class

- **Abstract Classes and Operations in UML**
- UML allows a class to be specified in an incomplete manner by omitting the definition of certain operations.
- An operation is said to be **abstract** when its signature is known but its implementation is not specified. In UML notation, the name of an abstract operation is written in *italics*.
- A class is said to be **abstract** when it defines at least one abstract operation, or when it inherits from a parent class that contains an abstract operation that has not yet been implemented. In UML, the name of an abstract class is also written in *italics*.



Exemple

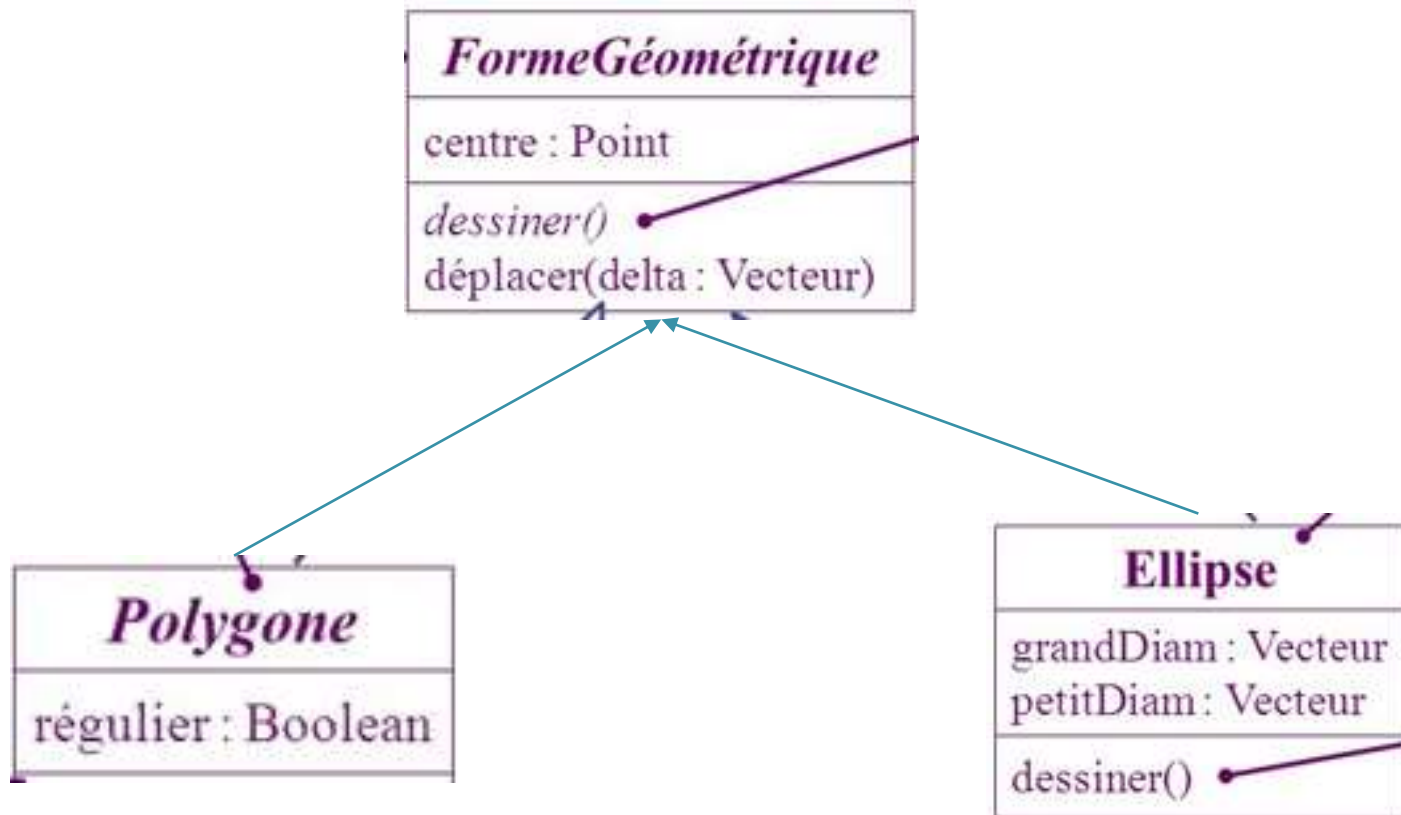
Forme_Geomtrique
centre : Point
dessiner () deplacer (delta:Vect)

<i>Forme_Geomtrique</i>
centre: Point
<i>dessiner ()</i> <i>deplacer (delta:Vect)</i>

Abstract Class

- It is the responsibility of the subclass to define the abstract operations.
- As long as at least one method remains abstract (even if it is inherited), the class itself must be considered abstract.
- It is not possible to instantiate an abstract class directly.

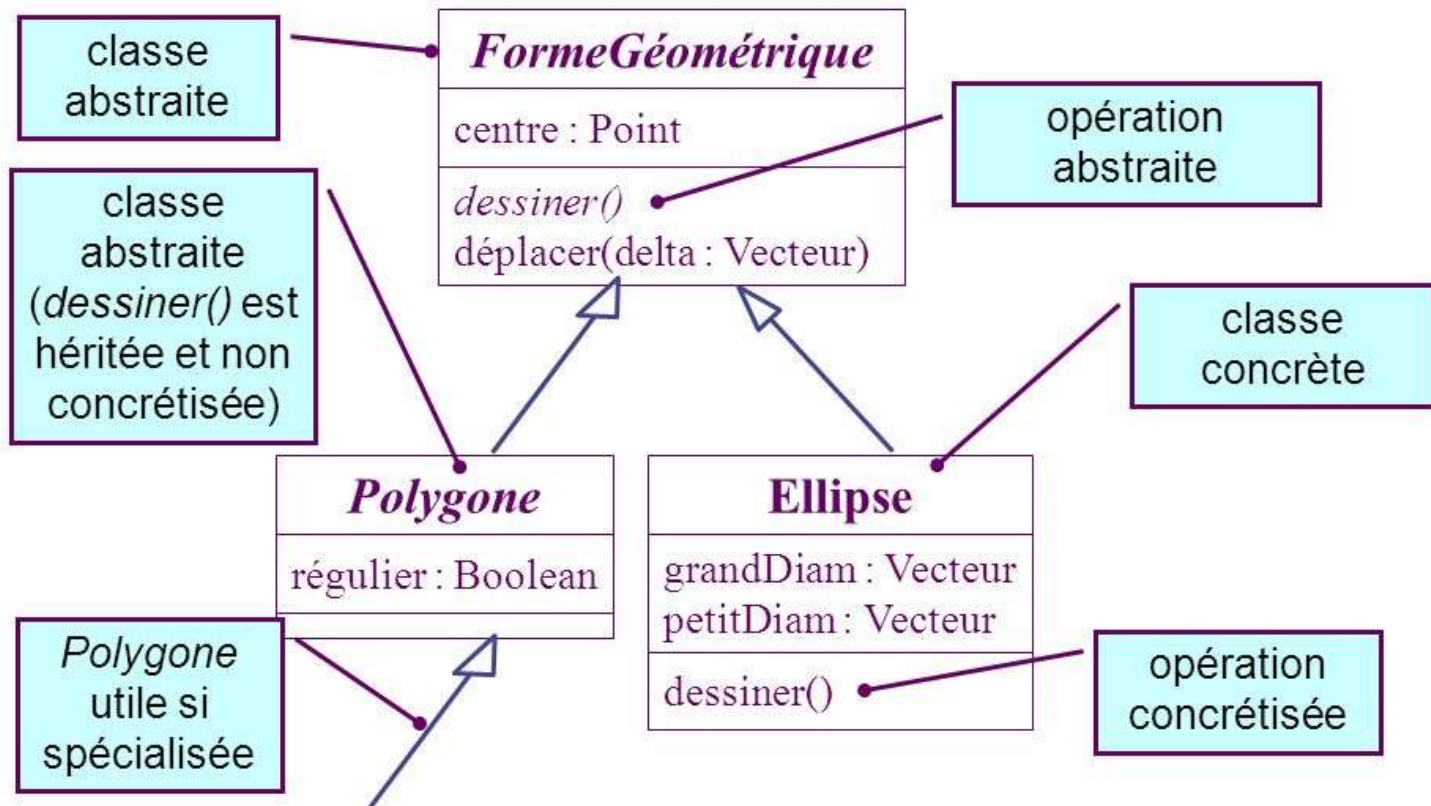
Exemple



Example of an Abstract Class

Modèle structurel

Classes abstraites



Interface

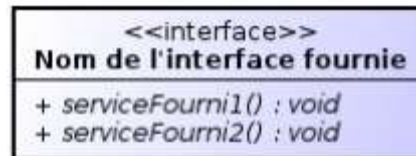
- **Definition**
 - **A collection of coherent abstract operations used to specify a service provided by a component or a class.**
- Interfaces are model elements that define sets of operations that other model elements, such as classes or components, are required to implement.
- Interfaces can be used in class diagrams and component diagrams to specify a contract between the interface and the implementing element.



Interface

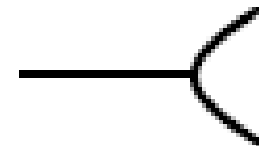
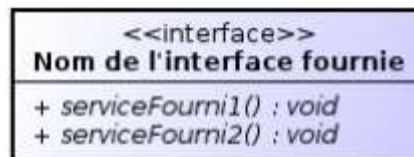
- **Definition**
 - A collection of operations used to specify a service provided by a component or a class.
- **Provided Interface**

Describes the services offered by a component to other components.



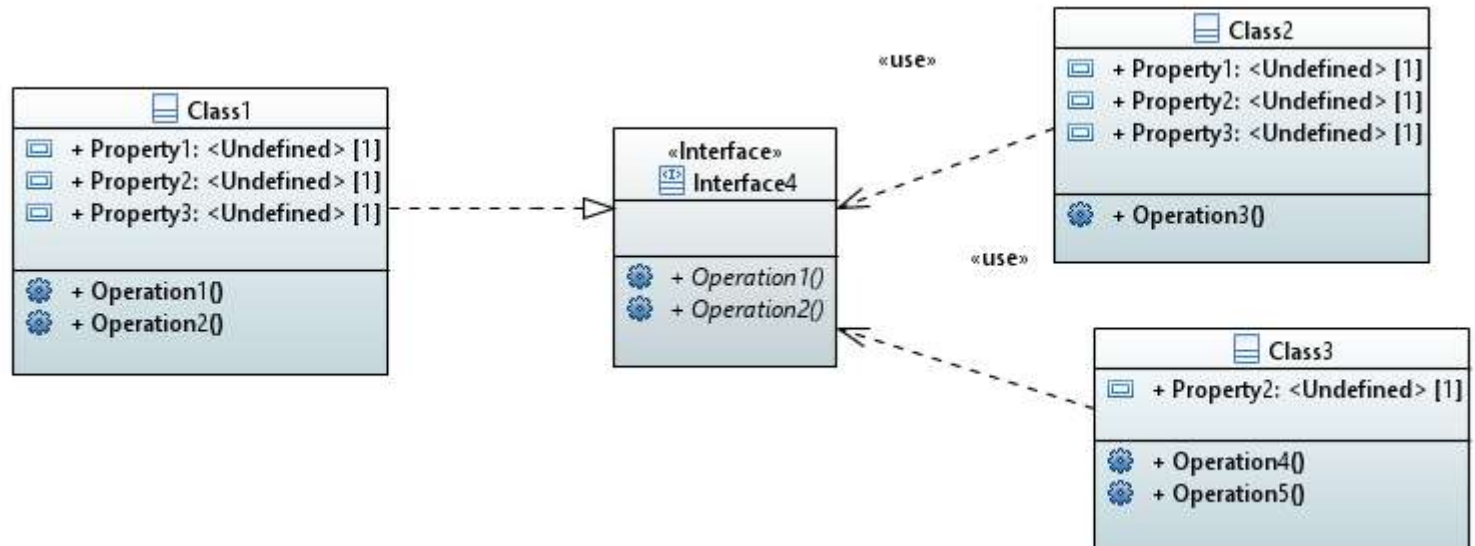
- **Required Interface**

Defines the services that a component needs in order to perform its functions.



Relationships Between a Class and an Interface

- **Realization (Implementation)**
- A realization relationship is used between an interface and a class that implements it (represented by a dashed inheritance arrow).
Classes implementing an interface must provide implementations for all the operations defined in the interface.
- **Dependency (use)**
- A class (the client of the interface) may depend on an interface (the required interface). This is represented by a dependency relationship with the stereotype « use ».



Exemple

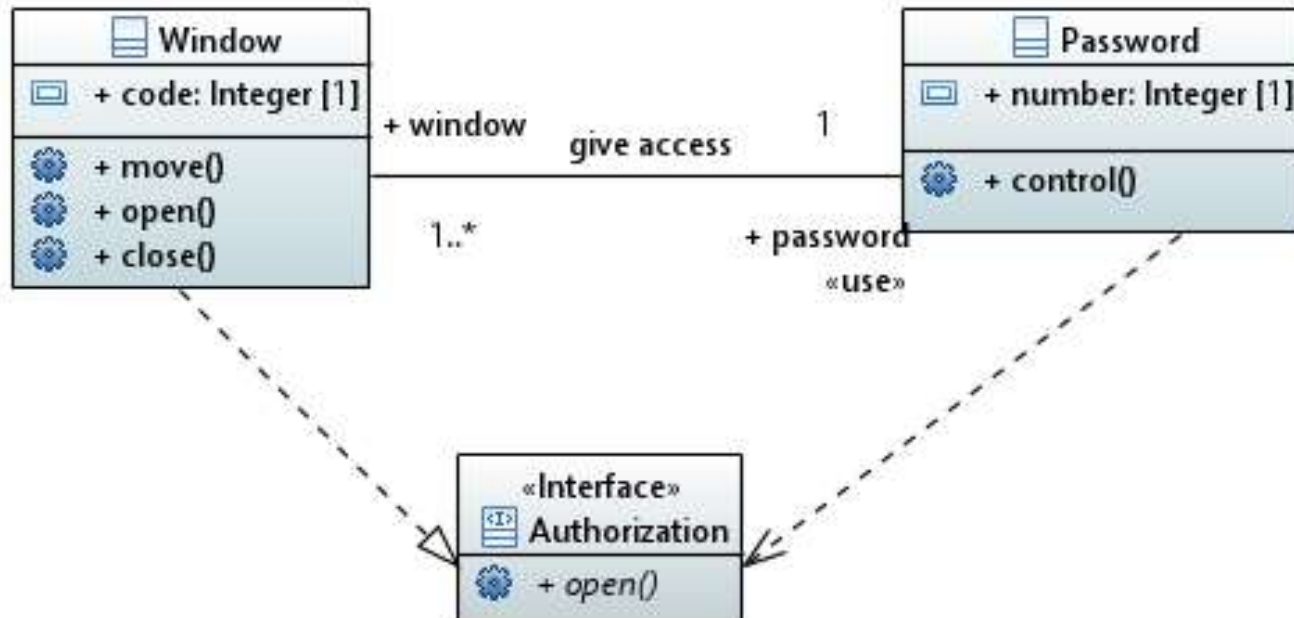
- An Authorization interface provides an operation open, which is implemented by a Window class. This operation is used within the control operation of the Password class. A Password grants access to a Window.

Exemple

- An **Authorization** interface provides an operation **open**, which is **implemented** by a **Window** class. This operation is **used** within the **control** operation of the **Password** class. A **Password** **grants access** to a **Window**.

Example

- An **Authorization** interface provides an operation **open**, which is **implemented** by a **Window** class. This operation is **used** within the **control** operation of the **Password** class. A **Password** grants access to a **Window**.





Component Diagram

Component Diagram

- Specifies the software architecture of a system.
- An abstract representation of a system, primarily expressed using software components interacting through connectors.
- Provides a static view of the system's implementation, showing design and realization choices.
- Describes the software components of the system and their dependencies.
- Primarily used for complex systems.
- Can be coupled with a deployment diagram to locate components physically.
- Suitable for component-based development (promoting reuse and component independence). Examples: EJB, CORBA, etc.
- **Elements (Nodes):** component, interface, port.
Relations (Links): dependency, realization.

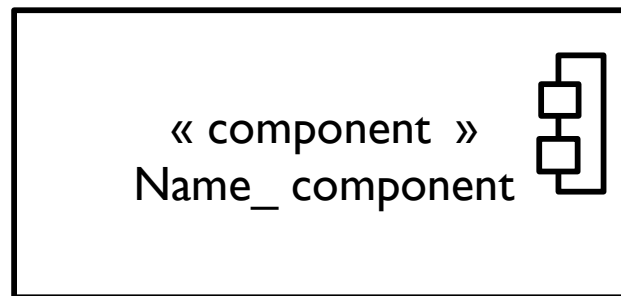
Component

- **Definition**

- A modular unit with well-defined interfaces (provided and required) that can be replaced within its environment or reused in another environment.

- **In Component-Based Development**

- The structuring of a system is based on the concept of a component, where a component is modeled throughout the development lifecycle and progressively refined until its execution and deployment as an artifact.

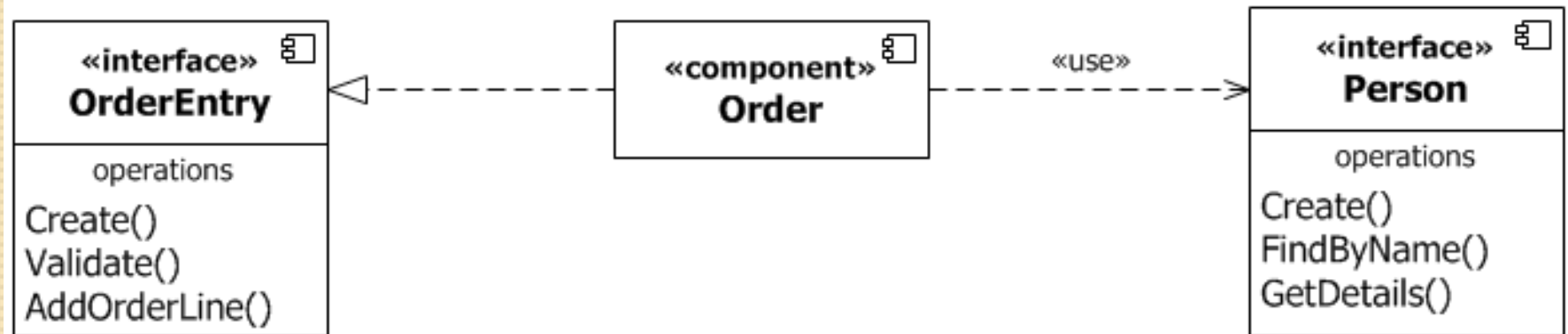


Example I

- The **Order** component has two interfaces: a **required interface**, **Person**, and a **provided interface**, **OrderEntry**. Each interface defines a set of operations.

Example I

The **Order** component has two interfaces: a **required interface**, Person, and a **provided interface**, OrderEntry. Each interface defines a set of operations.

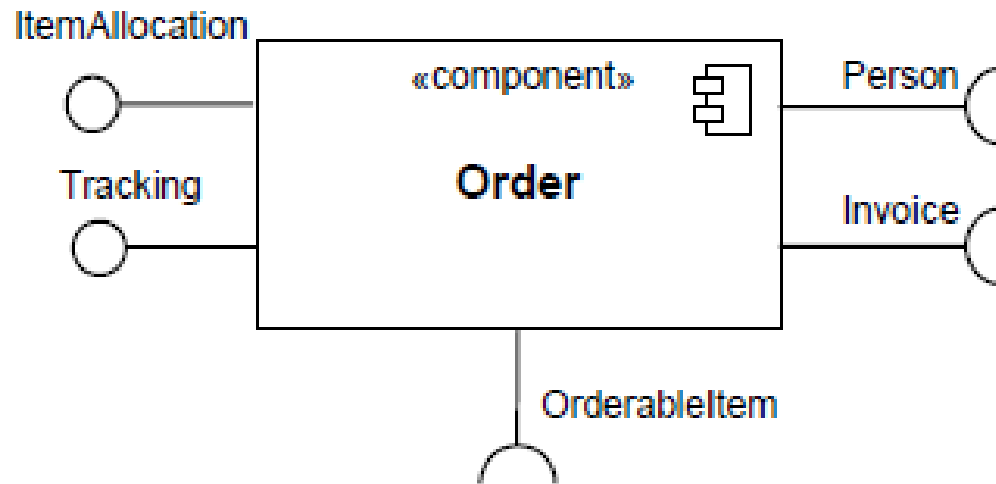


Example 2

- The **Order** component has five interfaces: three **required interfaces** — Person, PurchaseInvoice, and OrderedItem — and two **provided interfaces** — ItemAllocation and Tracking. Each interface defines a set of operations.

Example 2

- The **Order** component has five interfaces: three **required interfaces** — Person, PurchaseInvoice, and OrderedItem — and two **provided interfaces** — ItemAllocation and Tracking. Each interface defines a set of operations.

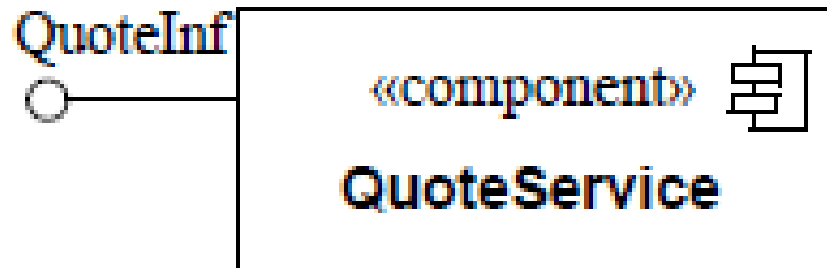


Example 3

- The **QuotationService** component has a **provided interface** that supplies quotation information.

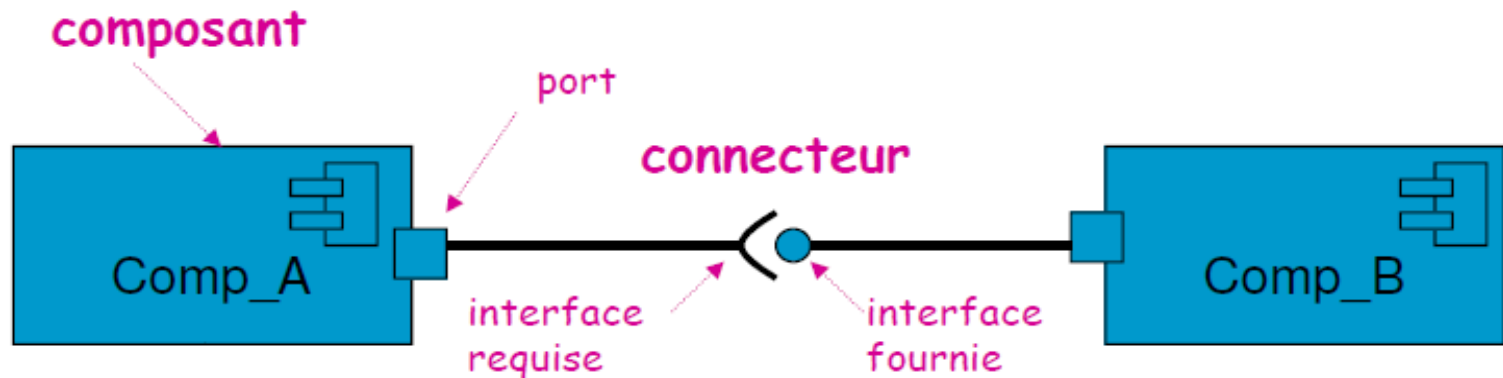
Example 3

- The **QuotationService** component has a **provided interface** that supplies quotation information.



Port

- A port is an interaction point (connection) between a component and its environment, representing the component's channels of interaction.



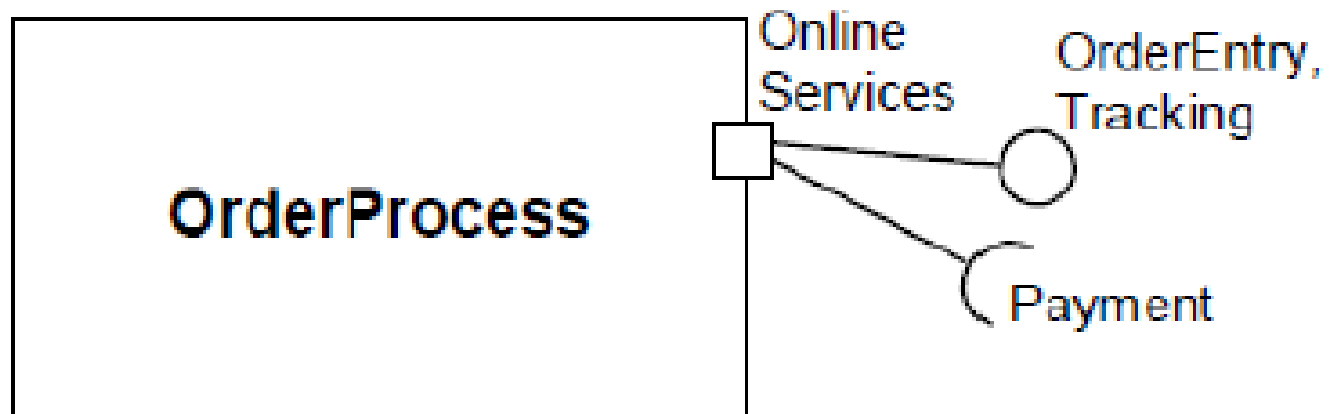
- It is associated with a **provided** or **required** interface. Multiple interfaces can be attached to the same port. Ports group incoming and outgoing messages and establish a communication protocol. A component can have multiple ports.

Example

- The **OnlineServices** port on the **OrderProcess** class has two **provided interfaces**, **OrderEntry** and **Tracking**, which are grouped on the same port, as well as one **required interface**, **Payment**.

Example

- The **OnlineServices** port on the **OrderProcess** component has two **provided interfaces**, OrderEntry and Tracking, which are grouped on the same port, as well as one **required interface**, Payment.



Characteristics of a Component

- An **autonomous unit**
- encapsulates the state and behavior of a number of classifiers,
- has one or more **provided** and/or **required interfaces** (potentially exposed via ports),
- specifies a **formal contract** of the services it provides to its clients and those it requires from other components or system services, in terms of provided and required interfaces,
- is encapsulated, and its dependencies are designed to be handled as independently as possible.

Characteristics of a Component

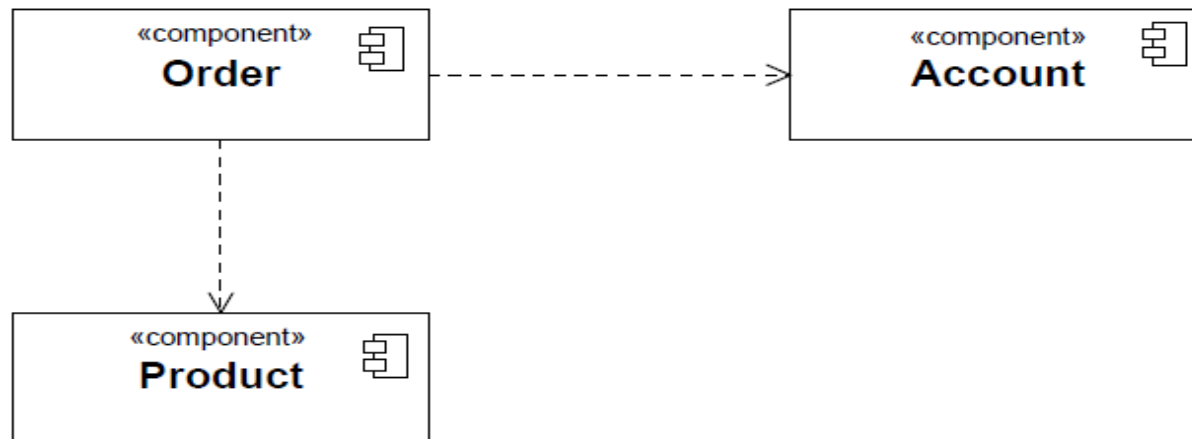
- **Reusable and Flexibly Replaceable**
- A component is a **substitutable unit** that can be replaced at design time or runtime by another component providing equivalent functionality, depending on the compatibility of its interfaces.
- It is implemented by **artifacts** that can be deployed and redeployed independently of updates to the existing system.
- A component is modeled throughout the **development lifecycle** and refined up to deployment.
- It can be realized by one or more artifacts, and in turn, each artifact can be deployed in its **runtime environment**.
- A **deployment specification** may define values that parameterize the execution of the component.

Type of component

- The **component package** supports the specification of both types of components:
 - **Logical components** (e.g., business components, process components)
 - **Physical components** (e.g., EJB components, CORBA components, COM+ and .NET components, WSDL components, etc.)
- It is expected that **profiles** will be developed for components specific to particular technologies and associated hardware and software environments.

Dependency

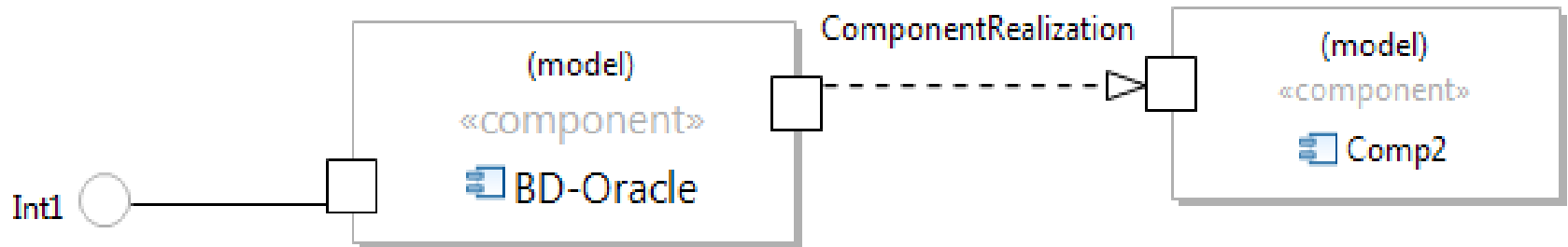
- Indicates that an implementation element of one component makes use of the services provided by the implementation elements of another component.



- DI) Usage Dependency (use)**
Indicates that a component requires the presence of the operations specified by an interface in order to execute.

Realization

- **Interface Realization (realize):**
Indicates that a component (the **supplier**) realizes or implements the operations specified by an interface (the **client**).
- **Component Realization:**
Indicates that a component (the **supplier**) realizes or implements the services of another component (the **client**).



Component representation

- **Black-Box Representation**

- This is an external view of a component that shows its **provided** and **required interfaces** without detailing the component's internal implementation. There are three forms:

- **Assembly Connector**
 - **Interface Connector**
 - **Compartment**

- **White-Box (Internal) Representation**

- This is an internal view of the component that describes its implementation using **classifiers** (classes, other components) that compose it. There are two forms:

- **Compartment**
 - **Dependency**

Black-Box Representation

- **Assembly Connector**

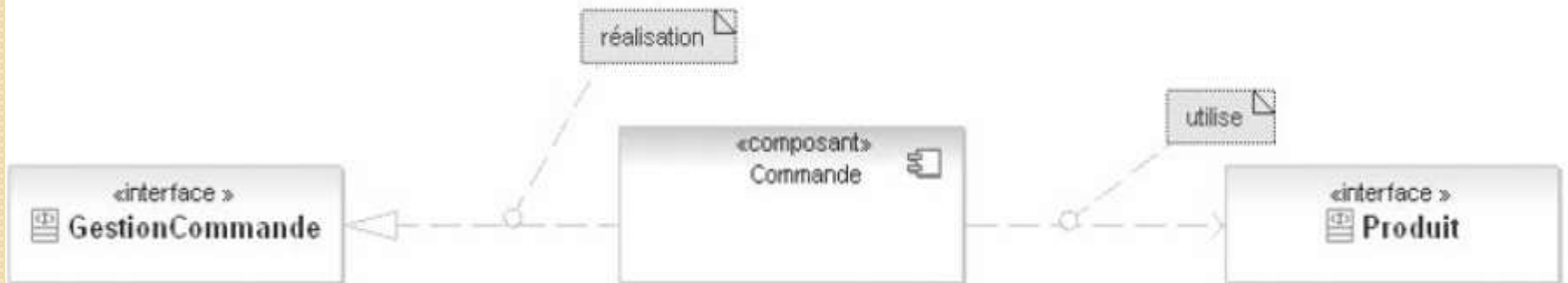
- A **provided interface** is represented by a line with a small circle, and a **required interface** is represented by a line with a half-circle.



Black-Box Representation

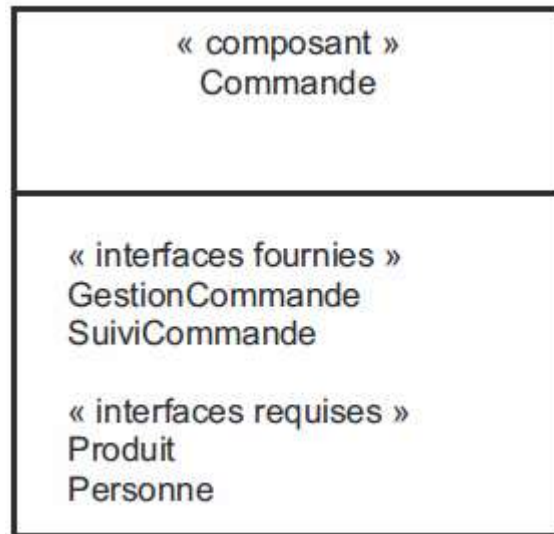
- **Interface Connector**

- Using **dependencies** and **interface realizations**:
- For a **provided interface**, it is a **realization relationship** from the component to the interface.
- For a **required interface**, it is a **dependency** with the keyword « use » from the component to the interface.



Black-Box Representation

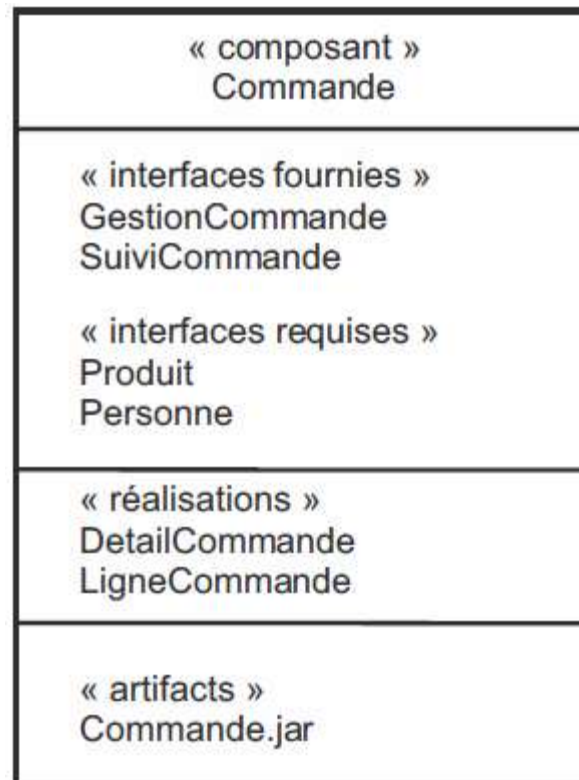
- **Compartment**
- Used to describe, in textual form, the **provided** and **required interfaces** within a second compartment.



Représentation

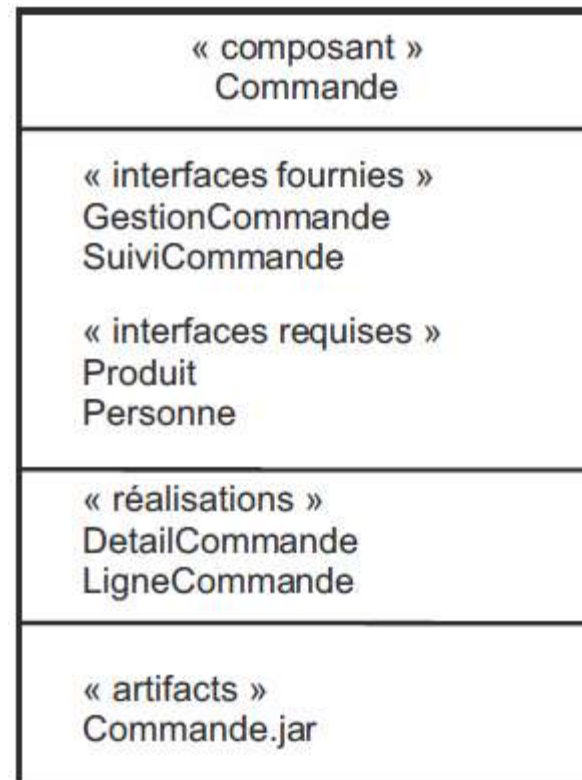
- **Compartment**

décrire sous forme textuelle les interfaces fournies et requises à l'intérieur d'un compartiment, les classificateurs (classes, autres composants) dans un autre compartiment, les artefacts (élément logiciel : jar, war, ear, dll,...) qui représentent physiquement le composant dans un dernier **compartiment**.



Representation

- **Compartment**
- Used to describe a component in textual form across multiple compartments:
- **One compartment** lists the **provided** and **required interfaces**.
- **Another compartment** shows the **classifiers** (classes, other components) that compose the component.
- **A final compartment** represents the **artifacts** (software elements such as JAR, WAR, EAR, DLL, etc.) that physically implement the component.

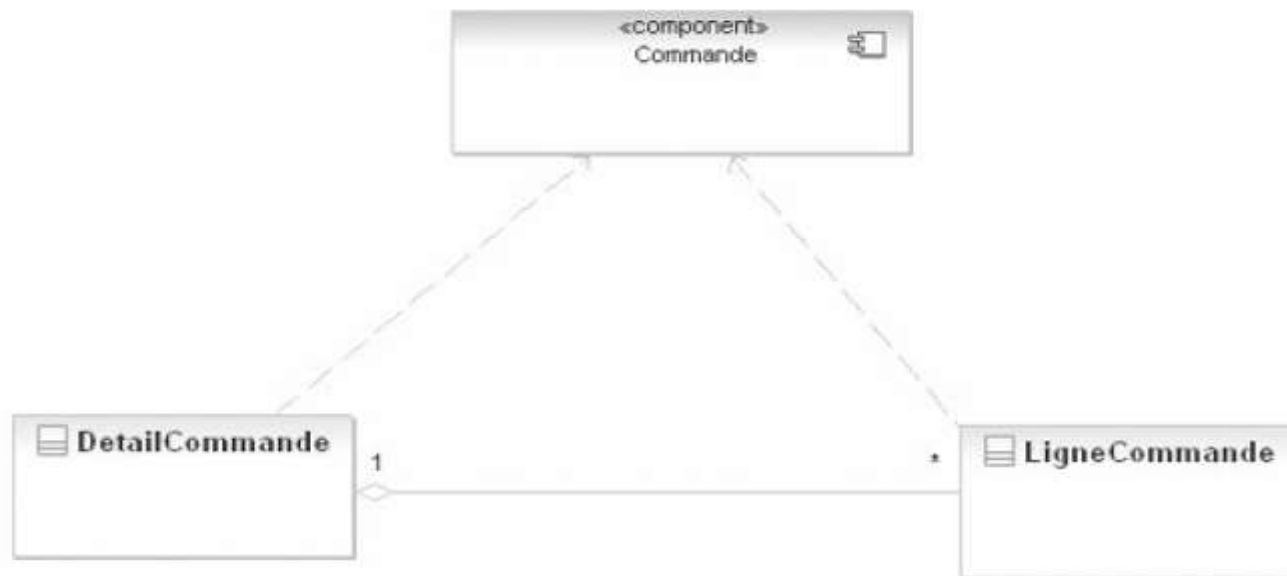


Representation

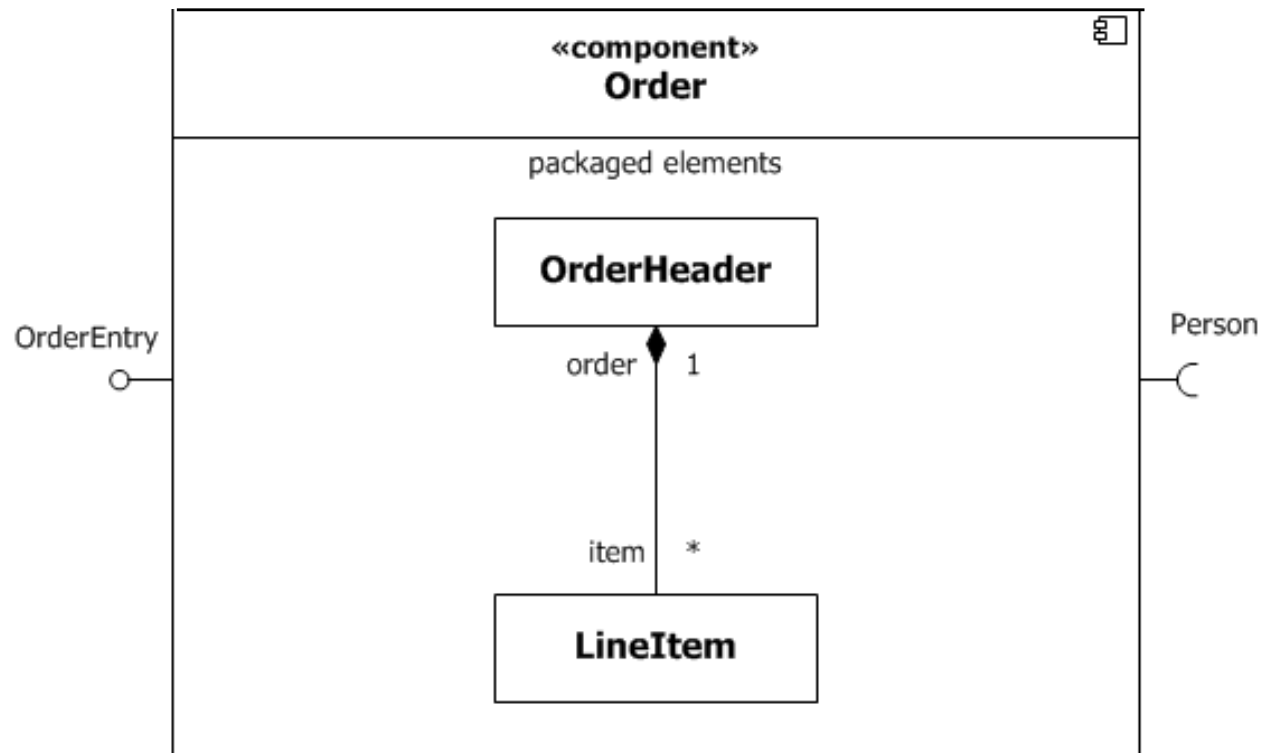
- **Dependency**

- The classifiers that compose a component are linked to it by a **dependency relationship**.

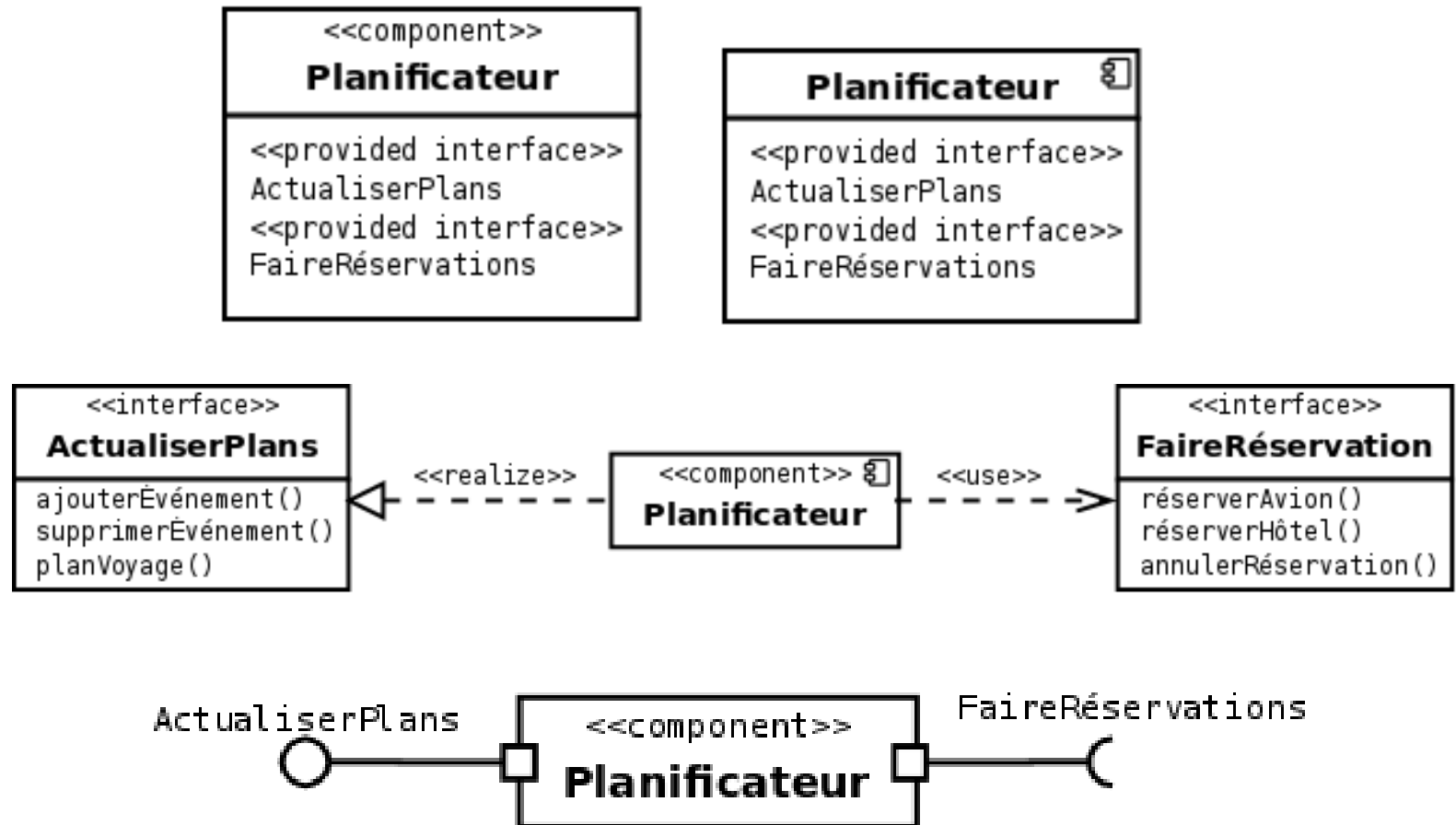
The relationships between classifiers (association, composition, aggregation) are also modeled. However, if these relationships are too complex, they can be represented in a **class diagram**.



Example



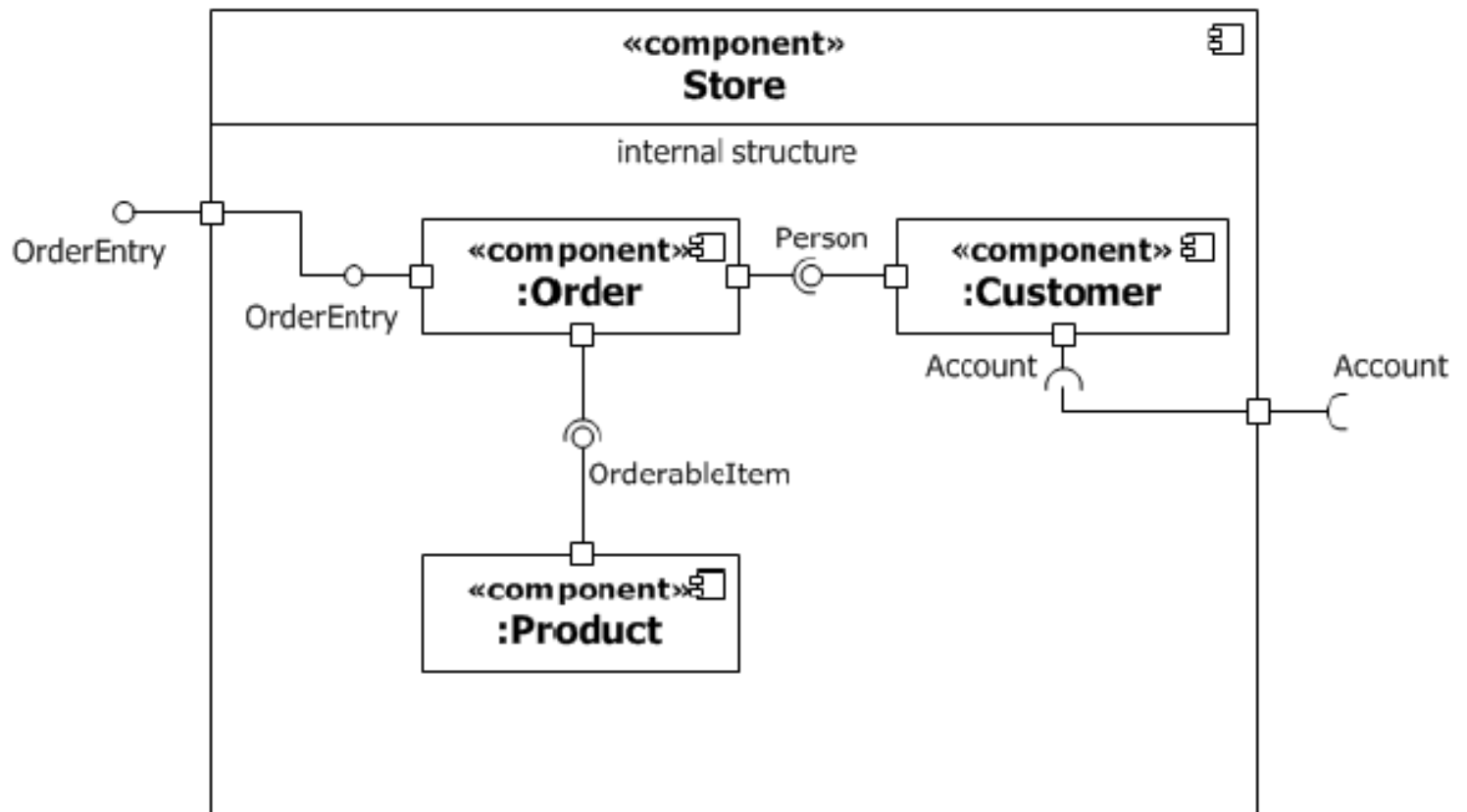
Exemple



Types of connectors

- There are two types of connectors:
 - **Assembly:** Indicates that the connector is an **assembly connector**.
 - **Delegate:** Indicates that the connector is a **delegation connector**.

Delegation et assembly





Merci de votre attention

Les diagrammes UML 2.5.1

